Uruchomienie, kompilacja i wdrożenie

Wersja 1.0.0, 3 listopada 2026. Sprawdzony host: macOS arm64; Node 24.11.0, npm 11.6.1. Lockfile przypina faktyczne wersje pakietów. Runtime gry nie wymaga Pythona.

Autor: kod źródłowy

1. Rozpakuj źródła do nowego folderu. Zainstaluj Node 24.x i npm.
2. W katalogu zawierającym package.json wykonaj npm ci, potem npm run dev.
3. Otwórz http://127.0.0.1:5173. Aplikacja wybiera lokalne demo.
4. npm run build tworzy frontend i serwer. npm start udostępnia je razem na porcie 3001.
5. Zatrzymaj serwer skrótem Ctrl+C. Zapis gry pozostaje w używanej przeglądarce.

Zwykłe sprawdzenia: npm run typecheck, npm run lint, npm test, npm run test:content. Przed npm run test:e2e wykonaj npx playwright install chromium ffmpeg. E2E uruchamia dev server automatycznie lub korzysta z już działającego. Zrzuty i raport JSON zapisuje w checks/.

Windows: odbiorca

Rozpakuj cały Naucz_Pilota_Windows.zip i uruchom Naucz Pilota.exe z rozpakowanego folderu. Zachowaj towarzyszące pliki i resources/. Node nie jest potrzebny. Gra zapisuje postęp w profilu lokalnym aplikacji.

Paczka nie ma podpisu cyfrowego. Windows może wyświetlić komunikat o nieznanym wydawcy; autor powinien sprawdzić integralność pobranej paczki i przetestować uruchomienie na osobnym komputerze. Nie wykonano jeszcze tego testu. Aktualna paczka ma pusty adres backendu i działa w jawnym demo; nie zapewnia live dla jury.

Konfiguracja modelu

Skopiuj .env.example do .env i uzupełnij AI_MODE=live, OPENAI_API_KEY, AI_MODEL. Plik przykładowy wskazuje lokalnie przetestowany gpt-5.4-mini. Dostęp modelu na własnym koncie sprawdź krótkim testem live. Klucz nie trafia do gry ani paczki desktopowej.

Dla Vite PUBLIC_ORIGIN=http://127.0.0.1:5173. Dla lokalnego npm start ustaw http://127.0.0.1:3001. W produkcji ustaw pełny origin HTTPS, NODE_ENV=production i trwały STATE_DB_PATH. Brak klucza/modelu jest jawnym błędem, bez podstawiania demo. Zmianę na demo wykonuje gracz.

npm run test:ai:live jest płatnym testem po ustawieniu klucza. Zatrzymaj wcześniej backend, aby nie uruchamiać drugiej instancji z tą samą bazą. Raport podaje prawdziwe metadane odpowiedzi; człowiek dopisuje ocenę semantyczną. Brak klucza daje NOT RUN i kod 2.

Backend na hostingu: przygotowana konfiguracja

Dostępny Dockerfile buduje aplikację na Node 24. Backend i frontend działają jako jedna domena. Kontener słucha na porcie 3001; reverse proxy zapewnia TLS. Zamontuj trwały wolumen /app/var i ustaw STATE_DB_PATH=/app/var/piko.sqlite. Przekaż sekrety jako zmienne serwera; nie kopiuj .env do obrazu. PUBLIC_ORIGIN ma odpowiadać adresowi publicznemu.

Uruchamiaj jedną instancję procesu i jeden trwały licznik SQLite. Nie skaluj przez niezależne kopie bazy. Aplikacja domyślnie nie ufa X-Forwarded-For; proxy musi też limitować żądania przy publicznym ruchu. Każda sesja ma 6 tur/min, globalny limit to domyślnie 200 żądań dostawcy/dzień UTC. Naprawa struktury zużywa kolejne żądanie; maksymalnie 2 na turę, 15 sekund łącznie. Limit wywołań nie gwarantuje kwoty rachunku.

Cookie web jest HttpOnly/SameSite=Strict, Secure w produkcji. Desktop trzyma token w main process, przyjmuje tylko zaufany origin HTTPS i blokuje przekierowania. W dystrybucji ustaw apiBaseUrl w apps/desktop/distribution.json, potem wykonaj npm run desktop:dist. Wyjątek HTTP 127.0.0.1 dotyczy tylko desktop:dev. Nie wdrożono publicznego endpointu w tym zadaniu.

Sesje, mała historia i dowody mają TTL 24h; usuwanie wygasłych rekordów wykonuje start nowej sesji. Globalny licznik jest trwały. Log produkcyjny zawiera identyfikatory techniczne, czas, model i kod błędu, bez pełnej rozmowy. Nie ma kont ani zewnętrznej analityki.

Budowanie dystrybucji

npm run desktop:dev buduje i otwiera Electrona. npm run desktop:dist przygotowuje minimalny staging i ZIP Windows x64. npm run release:check sprawdza PE, zawartość ASAR, publiczne dane i renderer. npm run release:prepare kopiuje materiały do folderu zgłoszeniowego i tworzy źródłowy ZIP z lockfile, bez .env, var, node_modules i lokalnego postępu.

Sprawdź ZIP na czystym Windows: start offline, M01, ponowny start z zapisem, próba live z przygotowanym HTTPS, jawna awaria sieci. Dopiero po takim teście oznacz Windows jako PASS. Publikacja, podpisy i wysłanie zgłoszenia pozostają po stronie autora.

Źródła implementacyjne

Oficjalne źródła sprawdzone podczas implementacji: https://developers.openai.com/api/docs/guides/structured-outputs?api-mode=responses ; https://vite.dev/guide/ ; https://www.electronjs.org/docs/latest/tutorial/security . Adapter używa Responses API, text.format JSON Schema strict i store:false. Nie deklarujemy jakości dialogu bez testów prawdziwego modelu.

Aktualizacja odbioru

Dockerfile został lokalnie zbudowany, a kontener przeszedł test zdrowia, braku sekretów i danych autora, użytkownika bez root oraz trwałości sesji SQLite po restarcie. npm run docker:build oraz npm run docker:smoke odtwarzają te sprawdzenia bez klucza AI.

Pierwszą próbę live można ograniczyć poleceniem npm run test:ai:live -- --case M01 --max-calls 4. Runner zatrzymuje się po pierwszym błędzie i zachowuje osobne NOT RUN dla pozostałych scenariuszy. Przejścia teach/transfer tej misji używają jednej sesji. Test Windows EXE i pilotaż mają protokoły w docs/16-windows-odbior.md i docs/15-pilotaz.md.

Odbiór lokalny live został wykonany: prompt 1.0.2 i model gpt-5.4-mini-2026-03-17, 22 tury oraz M04 w UI. Pozostają uwagi do semantyki, publiczny HTTPS i odbiór Windows. Narzędzie npx tsx tools/record-demo.ts --live świadomie wykonuje 2 tury płatne i nagrywa prawdziwą M04; brak flagi --live wybiera jawne demo. Nie uruchamiaj nagrania ani testów płatnych w zwykłym npm test.